

Your Project, Explained Simply

A friendly, no-jargon walkthrough of what you are building, how it works, and what every technical word means.

This guide assumes you know a little Python and nothing else. Every hard word is explained the moment it appears, and again in a glossary at the end. Read it once, top to bottom, and you should be able to explain your whole project to a friend.

Codebase-Wide Version Migration & Refactoring Agent

Prepared for: Srinivas RC · Guide: Dr Nimrita Koul · Centre for AI & ML

Version 1.0 · 24 August 2026

What's inside

1. The project in one breath
2. The name, decoded word by word
3. The problem you are solving (with a tiny example)
4. Why this is genuinely hard
5. How your agent works — the five steps
5. What the project looks like (the folders)
7. The toolbox — every keyword in simple words
3. How you'll know it works — the report card
9. A complete worked example, start to finish
9. What "good" looks like & what to do next

SECTION 1

The project in one breath

You are building a robot software-engineer that upgrades an entire codebase by itself when a tool it depends on changes.

Software is built on top of other people's code, called **libraries**. Libraries release new versions. Sometimes a new version **breaks** the old way of doing things — code that used to work now fails. Fixing this by hand across a big project is slow, boring, and easy to get wrong.

Your project is an **agent** (a program that acts on its own) that does this fix automatically. You tell it, in plain words, "*upgrade this project from the old version to the new version.*" It then reads the whole project, finds every place that will break, fixes them in a safe order, runs the project's tests to check its work, and — this is the clever part — if it breaks something, it reads the error, understands it, and fixes it. It keeps going until everything passes.

THINK OF IT LIKE THIS

Imagine a car company changes the design of one engine part. A **robot mechanic** takes your whole car, finds every part that touched the old engine part, swaps them in the right order, then **starts the engine to test it**. If a warning light comes on, it reads the light, figures out what it did wrong, fixes that too, and tests again — until the car runs perfectly. Your project is that robot mechanic, but for code.

The important word is **test**. Your agent is not a lazy "find and replace." It *checks its own work* by running the project's tests after every change, and repairs its own mistakes. That self-checking, self-repairing loop is the heart of the project.

SECTION 2

The name, decoded

The full name is "**Codebase-Wide Version Migration & Refactoring Agent.**" It sounds heavy. It's four simple ideas stuck together.

Word	What it really means
Codebase-Wide	Across the <i>whole</i> project — every file, not just one. A "codebase" is just "all the code in a project."
Version Migration	Moving code from an old version of a tool to a new one. "Migration" = moving from A to B.
Refactoring	Changing how code is written without changing what it does. Like rephrasing a sentence to say the same thing more correctly.
Agent	A program that makes its own decisions and takes actions to reach a goal — not a fixed script.

IN ONE SENTENCE

"A self-driving program that upgrades all the code in a project from an old tool version to a new one, and rewrites it correctly."

Tip: you can give it a short codename in conversation (some students call theirs "MRA" — Migration & Refactoring Agent). The formal name stays the same.

SECTION 3

The problem you are solving

Here is a real, tiny example of a version break — the kind your agent fixes.

Python used to have a common way to get the current time: `datetime.utcnow()`. In newer Python, that way is **deprecated** (marked "don't use this anymore, it causes bugs"). The correct new way is `datetime.now(timezone.utc)`.

Now imagine a small project with two files. **File 1** creates a timestamp:

```
# core.py — the OLD way
from datetime import datetime

def make_timestamp():
    return datetime.utcnow() # deprecated!
```

File 2 uses File 1:

```
# report.py — depends on core.py
from core import make_timestamp

def log_event(name):
    return f"{name} at {make_timestamp()}"
```

To upgrade, the agent must change `core.py` to the new way — and add a small import:

```
# core.py — the NEW, correct way
from datetime import datetime, timezone

def make_timestamp():
    return datetime.now(timezone.utc)
```

WHY THIS IS MORE THAN FIND-AND-REPLACE

The agent can't just swap text. It has to **notice that** `report.py` uses `core.py`, add the missing `timezone` import in the right file, and then **run the tests** to prove nothing else broke. On a real project there are dozens of files like `report.py`, not one.

SECTION 4

Why this is genuinely hard

Three things make this a real engineering challenge — not a weekend script.

1. It spreads across files

Change one shared piece of code and you can break ten files that use it. The agent must **map who-uses-what before it edits**, or it fixes one file and breaks three others.

EXAMPLE

If it edits `core.py` but forgets that `report.py` and `emailer.py` both call it, those two files might now fail.

2. It's a long job

A real upgrade touches dozens of files over hundreds of small steps. The agent cannot hold the entire project in its head at once, so it needs a **compact memory** of what is done, half-done, and still to do.

3. Some breaks are invisible until you test

The scariest breaks don't crash. The code still runs — it just **behaves differently**. Only running the tests reveals the problem.

REAL EXAMPLE OF AN INVISIBLE BREAK

When moving from the `requests` library to `httpx`, a setting called "follow redirects" quietly flips from **on** to **off**. The code runs with no error — but now it silently stops following web redirects. A human eye misses this. A **test** catches it.

THE BIG IDEA

Because some breaks are invisible, the agent **must run the tests** and **must be able to fix what the tests reveal**. That is why the "test and self-correct" loop is 80% of the value of this project.

SECTION 5

How your agent works — the five steps

Your agent runs the same five steps in a loop: MAP → PLAN → EDIT → TEST → CORRECT. Keep the robot-mechanic picture in mind.

1 MAP — look before you touch

The agent reads the whole project and makes a list of **every place that will break**, plus a chart of **which files depend on which**. Nothing is edited yet.

Mechanic: before touching anything, the robot inspects the whole car and marks every part connected to the old engine part.

2 PLAN — decide a safe order

Using the dependency chart, the agent decides **what order to fix things in** so it never breaks file B while fixing file A. It groups the edits into small batches.

Mechanic: the robot plans which parts to swap first so the car is never left half-connected.

3 EDIT — make the change

The agent edits the next batch of files. For simple, mechanical changes it uses an exact tool (no guessing). For tricky changes it asks an AI model. It carefully keeps your formatting and comments intact.

Mechanic: the robot swaps the parts — using a precise wrench for standard bolts, and judgement for the fiddly bits.

4 TEST — check the work

The agent runs the project's **tests** inside a safe, sealed box (so a mistake can't harm your real computer). The tests come back either all-green (pass) or with failures.

Mechanic: the robot starts the engine and reads the dashboard.

5 CORRECT — fix its own mistakes

If a test fails, the agent **reads the error message**, works out which change caused it, makes a repair, and goes back to TEST. It repeats until everything passes — or, if it keeps failing the same way, it gives up gracefully and asks a human.

Mechanic: a warning light comes on; the robot reads it, fixes the cause, and re-tests. This loop is the clever heart of the whole project.

THE LOOP IN ONE LINE

MAP what will break → **PLAN** a safe order → **EDIT** a batch → **TEST** it → if it fails, **CORRECT** and test again → when everything passes, done.

SECTION 6

What the project looks like

Your project is a set of folders. Here's what each one is *for*, in plain words.

Folder / file	What lives here (plain words)
README.md	The front page. The first thing anyone reads. "What is this and how do I start it?"
CLAUDE.md	The rulebook for any AI helper working on the code. "Here's how we do things, here's what you must never do."
.env	Your secret keys , kept private (never shared, never uploaded). Like your house keys.
docs/	All the planning documents (charter, requirements, design, evaluation). The project's paperwork.
src/mra/	The actual agent's code. The engine. Inside are the five steps (map, plan, edit, test, correct).
corpus/	Your practice projects — small codebases with known problems the agent learns to fix, plus the "correct answer" to grade against.
runs/	The results of each time you run the agent: what it changed, a diary of its decisions, and its score.
tests/	Tests for the agent itself (to make sure your agent's own code works).

THINK OF IT LIKE A WORKSHOP

`src/` is the robot itself. `corpus/` is the shelf of practice cars. `runs/` is the logbook of every job it did. `docs/` is the manual. `.env` is the locked drawer with the keys.

SECTION 7

The toolbox — every keyword in simple words

These are the technical words used in your project. Each card has a plain meaning, a tiny analogy, and how it's used here.

IMPORTANT: LANGCHAIN VS LANGGRAPH

You said "LangChain," but your project actually uses **LangGraph**. They're related, and it's easy to mix them up:

LangChain is a big, general toolkit for building apps with AI models (like a whole hardware store).

LangGraph is one specific tool *from the same makers*, built for agents that follow a clear set of steps in a loop (like one specialist power-tool from that store). Your five-step loop

(MAP→PLAN→EDIT→TEST→CORRECT) is exactly what LangGraph is designed for. So: **LangGraph is the right tool, and it's the one your project uses.**

Agent core idea

A program that decides its own next action to reach a goal, instead of following a fixed script.

Analogy: a GPS that re-routes you when you miss a turn, versus a printed list of directions that can't adapt.

In your project: the whole thing is an agent — it decides what to edit, when to test, and how to fix failures.

LangGraph framework

A tool for building agents as a clear diagram of steps ("do this, then that; if it fails, loop back").

Analogy: a flowchart that actually runs.

In your project: it runs and connects your five steps and handles the "if the test fails, go back and correct" loop.

LLM (Large Language Model) the "brain"

An AI trained on huge amounts of text and code that can read, write, and reason in words and code.

Analogy: a very well-read assistant who can write and understand code, but sometimes guesses wrong.

In your project: used for the tricky edits and for reading error messages to figure out fixes.

DeepSeek (V4-Pro / V4-Flash) the specific AI

The particular LLM your project uses. "Pro" is the smart, pricier one; "Flash" is the cheap, fast one.

Analogy: hiring a senior engineer for hard problems and an intern for the quick, cheap tasks.

In your project: Pro does edits and fixes; Flash does quick summaries and sorting. Using the cheap one where possible saves money.

AST & libcst reading code

An **AST** ("Abstract Syntax Tree") is code turned into a structured tree a program can understand. **libcst** is the tool that builds this tree and lets you edit code precisely while keeping formatting and comments.

Analogy: instead of treating a sentence as loose letters, you understand its grammar — subject, verb, object — so you can edit it correctly.

In your project: libcst is how the agent finds exact places to change and makes clean, exact edits.

Codemod exact edit

A pre-written, exact code transformation ("wherever you see X, turn it into Y"). No AI guessing.

Analogy: a stencil — same precise shape every time.

In your project: used for simple mechanical changes; it's exact, free, and reliable, so the AI is saved for the hard cases.

Dependency graph & networkx who-uses-what

A **dependency graph** is a map of which files rely on which other files. **networkx** is the Python tool that builds and studies this map.

Analogy: a family tree, but for code — showing who depends on whom.

In your project: the agent uses this map to edit in a safe order (Step 2, PLAN).

Topological sort safe order

A way to order tasks so that anything you depend on comes first. It turns the dependency map into a to-do list.

Analogy: you put on socks before shoes — never the other way around.

In your project: it decides which files to fix first so nothing is edited out of order.

Test suite, pytest & ruff checking

A **test suite** is a set of automatic checks that prove code works. **pytest** runs them. **ruff** checks code style and catches obvious mistakes.

Analogy: the exam paper (tests), the examiner (pytest), and a spell-checker (ruff).

In your project: after each edit, pytest + ruff tell the agent whether it succeeded (Step 4, TEST).

Docker & the sandbox safety

Docker creates a **sandbox**: a sealed, disposable mini-computer where code runs isolated from your real machine.

Analogy: a fireproof test chamber — if something explodes, your real computer is untouched.

In your project: all the agent's edits and tests run inside the sandbox, so a bad change can never damage your laptop.

git, patch & diff tracking changes

git tracks every change to code. A **diff** shows exactly what changed (old vs new). A **patch** is a file containing those changes, ready to apply.

Analogy: "track changes" in a Word document, saved as a single file you can hand over.

In your project: the final "here's the whole upgrade" is delivered as one patch file, and git lets the agent undo a bad batch.

State machine & checkpointer memory

A **state machine** is a system that is always in one clear step and moves to the next by rules. A **checkpointer** saves the current step and memory so nothing is lost.

Analogy: a board game where your piece is always on one square, and you photograph the board after every move.

In your project: LangGraph is the state machine; the checkpointer saves progress and becomes the agent's decision diary.

Token cost unit

A token is a small chunk of text (roughly $\frac{3}{4}$ of a word). AI models charge by the number of tokens they read and write.

Analogy: words on a taxi meter — the more the AI reads and writes, the more it costs.

In your project: you measure tokens to keep the agent cheap and efficient (one of your scores).

JSON & schema data shape

JSON is a simple text format for structured data (lists and labelled values). A **schema** is the agreed shape that JSON must follow.

Analogy: a form with fixed fields — name, date, amount — that must always be filled the same way.

In your project: the agent's memory and results are stored as JSON in a fixed schema, so nothing is ambiguous.

How you'll know it works — the report card

Your agent is graded on three numbers. Think of it as a report card.

Score	Plain meaning	Good result
M1 — Completeness	Of all the places that needed fixing, how many did the agent fix correctly? (And did it avoid changing things it shouldn't?)	High = it found and fixed everything.
M2 — Test pass rate	After the agent finished, how many of the project's tests pass?	100% = a fully working upgrade.
M3 — Cost / effort	How much did it "spend" (AI tokens and steps) to do the job?	Lower = more efficient.

THE HEADLINE EXPERIMENT

The most convincing result for your report and viva: run the agent **with** the self-correcting loop, then **without** it, and show the test-pass score drops a lot without it. That proves the loop — the hard part — is what makes the project work.

SECTION 9

A complete worked example, start to finish

Let's follow the agent through one real job — upgrading the `datetime.utcnow()` project from Section 3.

You type: `run --repo task01_datetime --task "utcnow → now(timezone.utc)"`

1 MAP

The agent scans the project. It finds one broken spot: `datetime.utcnow()` in `core.py`, line 4. It notes that `report.py` depends on `core.py`. It writes this down as its map.

2 PLAN

Since `report.py` depends on `core.py`, the agent decides to fix `core.py` first, together with checking `report.py`. One small batch.

3 EDIT

This is a simple, known change, so the agent uses an exact codemod: it turns `datetime.utcnow()` into `datetime.now(timezone.utc)` and adds `timezone` to the import line. Formatting stays clean.

4 TEST

Inside the sandbox, the agent runs pytest. Suppose one test fails: it expected a timezone-aware time, and an early edit missed the import, causing a `NameError: timezone is not defined`.

5 CORRECT

The agent reads the error, sees the missing `timezone` import, adds it, and returns to TEST.

4 TEST (again)

pytest runs again. All green. Every test passes.

WHAT YOU GET AT THE END

In `runs/<id>/` you find three things: the **patch** (the whole upgrade as one file), the **trajectory** (a diary: "found 1 site, edited core.py, test failed on missing import, fixed it, tests passed"), and the **score** (Completeness 100%, Tests 100%, and how many tokens it used).

THE MECHANIC FINISHED THE JOB

The robot mechanic swapped the part, started the engine, saw one warning light, fixed the loose wire behind it, restarted, and handed you the keys with a full service report.

SECTION 10

What "good" looks like & what to do next

A good version of this project is small, working, and honest — not big and half-built.

- **It runs end-to-end early.** Even a tiny agent that fixes one file and tests it beats a huge one that never runs.
- **The self-correcting loop actually works.** This is the part that impresses. Protect time for it.
- **It's safe.** Everything runs in the sandbox; your laptop is never at risk.
- **It's honest.** Your report shows where it failed, not just where it won. That's what makes it real science.
- **You can explain it simply.** If you can describe this project to a friend using the mechanic story, you understand it — and that's exactly what a viva panel and an interviewer want to hear.

YOUR ONE NEXT STEP

Build **practice project #1** — the little `datetime` example from Section 3, with two files and a working test. It needs no AI at all. It becomes the first thing your agent learns to fix, and the answer key you grade it against. Everything else in the project stands on this small piece, so build it first.